

■ COMPLETE NOTES - TOPIC 07

CSS Display Property

Basic se Advanced tak - element layout behavior ka complete system

Web Development Course - Shauray

block

inline

inline-block

none

flex

grid

flow-root

contents

table

list-item

visibility vs opacity

outer / inner display

Display Property Kya Hai?

Browser ko batana ki element page par kis tarah ka box banayega

SIMPLE DEFINITION

display CSS property decide karti hai ki HTML element layout mein kaise behave karega: kya woh full line lega, text ke saath same line mein rahega, children ko flex/grid layout mein arrange karega, ya bilkul render hi nahi hoga.

Har HTML element browser ke andar ek box banata hai. **display** us box ka bahar ka behavior aur andar ka layout system control karta hai. Isliye display property CSS layout ka foundation hai. Agar display samajh aa gaya, to Flexbox, Grid, navbar, cards, hidden elements, responsive menus aur page structure easy ho jata hai.

basic syntax

```
selector {  
  display: block;  
}  
  
.nav {  
  display: flex;  
}
```

■ Display Property Ka Main Kaam

Control	Meaning	Example
Outer behavior	Element surrounding elements ke saath kaise behave karega	block, inline, inline-block
Inner layout	Element apne children ko kaise arrange karega	flow, flex, grid, table
Rendering	Element visible/rendered rahega ya layout se remove hoga	none, contents

■ Default Display Values

HTML elements ke paas default display values hoti hain. Example: `div` block hota hai, `span` inline hota hai, `ul` block hota hai aur `li` list-item hota hai.

Element	Default Display	Behavior
<code>div</code> , <code>p</code> , <code>h1-h6</code> , <code>section</code>	block	New line start, full available width leta hai
<code>span</code> , <code>a</code> , <code>strong</code> , <code>em</code>	inline	Text ke flow mein same line par rehta hai
<code>img</code> , <code>input</code> , <code>button</code>	inline-block / inline-level	Same line mein aa sakte hain but size control possible hota hai
<code>ul</code> , <code>ol</code>	block	List container full width leta hai
<code>li</code>	list-item	Marker/bullet ke saath list item box banata hai

■ **Golden Start:** `display` ka value badalte hi element ka box model behavior change ho sakta hai. Isliye layout bug debug karte time pehle element ka computed display check karo.

Block vs Inline vs Inline-Block

CSS layout ka sabse important basic comparison

■ 1. display: block

`block` element new line se start hota hai aur by default parent ki full available width le leta hai. Width, height, margin, padding properly apply hote hain.

BLOCK VISUAL DEMO

Block 1

Block 2

Block 3

● ● ● block example

```
.box {  
  display: block;  
  width: 300px;  
  height: 120px;  
  margin: 20px auto;  
}
```

■ 2. display: inline

`inline` element text flow ke andar rehta hai. Yeh new line force nahi karta. Is par `width` aur `height` ka effect normally nahi hota. Horizontal margin/padding work karte hain, but vertical spacing layout ko expected tarah push nahi karta.

INLINE VISUAL DEMO

Normal text ke beech inline 1 inline 2 inline 3 same line par flow karte hain.

inline example

```
a {  
  display: inline;  
  color: hotpink;  
  padding-inline: 8px;  
}
```

3. display: inline-block

`inline-block` dono ka mix hai. Element same line mein aa sakta hai, but width, height, margin aur padding block element ki tarah control ho sakte hain. Buttons, badges, chips, small cards aur nav links mein kaafi useful hai.

INLINE-BLOCK VISUAL DEMO

110px

110px

110px

Comparison Table

Feature	block	inline	inline-block
New line	Yes	No	No
Default width	Full available width	Content width	Content width
Width/height	Works	Mostly ignored	Works
Best use	Sections, cards, containers	Text links, spans, emphasis	Buttons, badges, nav items

■ **Practical Tip:** Anchor tag ko button jaisa banana ho to usually `display: inline-block` ya `display: inline-flex` use karo, taaki padding aur dimensions cleanly work karein.

display: none aur Hiding Techniques

Element ko hide karne ke different tareeqe aur unka layout par effect

KEY IDEA

`display: none` element ko page rendering tree se remove kar deta hai. Element visible nahi hota aur layout mein space bhi reserve nahi karta.

● ● ● display none

```
.modal {  
  display: none;  
}  
  
.modal.is-open {  
  display: block;  
}
```

■ display none vs visibility hidden vs opacity 0

Property	Visible?	Space Reserved?	Clickable?	Common Use
<code>display: none</code>	No	No	No	Completely remove from layout
<code>visibility: hidden</code>	No	Yes	No	Hide but keep layout space
<code>opacity: 0</code>	No visually	Yes	Yes, unless pointer-events none	Fade animation

■ Animation Important Point

`display` ko smoothly transition nahi kar sakte like opacity. Agar fade-in/fade-out chahiye, to `opacity`, `visibility`, `transform`, aur sometimes `pointer-events` combine karna better hota hai.

```
.dropdown {  
  opacity: 0;  
  visibility: hidden;  
  pointer-events: none;  
  transform: translateY(8px);  
  transition: 200ms ease;  
}  
  
.menu:hover .dropdown {  
  opacity: 1;  
  visibility: visible;  
  pointer-events: auto;  
  transform: translateY(0);  
}
```

■ **Accessibility Note:** Important content ko sirf visually hide karke leave mat karo. Screen readers, keyboard focus aur tab order par effect samajhna zaroori hai, especially menus, modals aur form errors mein.

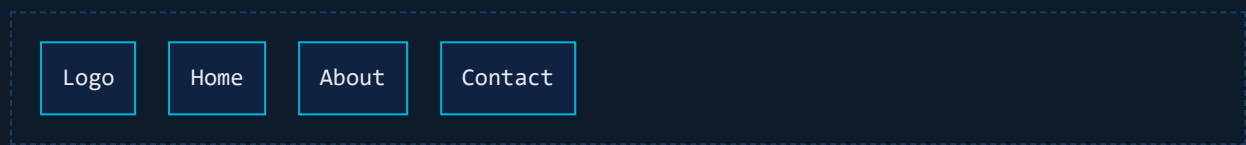
display: flex aur inline-flex

One-dimensional layout system for rows and columns

DEFINITION

`display: flex` element ko flex container bana deta hai. Uske direct children flex items ban jaate hain. Flexbox ek direction mein layout arrange karne ke liye best hai: row ya column.

FLEX VISUAL DEMO



navbar flex

```
.navbar {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 1rem;  
}
```

flex vs inline-flex

`flex` container outside se block-level behave karta hai. `inline-flex` outside se inline-level behave karta hai, but inside children flex items hi rehte hain.

Value	Outer Behavior	Inner Layout	Use
<code>display: flex</code>	Block-level container	Flex formatting context	Navbar, card layout, page sections
<code>display: inline-flex</code>	Inline-level container	Flex formatting context	Buttons with icon, badges, chips

■ Common Flex Use Cases

GOOD FOR

Centering items.
Navbar links in a row.
Icon + text buttons.
Equal height cards in one row.
Sidebar + content simple layouts.

REMEMBER

Flex affects only direct children.
Main axis depends on flex-direction.
gap works in modern browsers.
flex-wrap controls wrapping.
align-items controls cross axis.

■ **Centering Shortcut:** Single item center karna ho to parent par `display: grid; place-items: center;` bhi clean option hai. Multiple row/column control chahiye to flex useful hai.

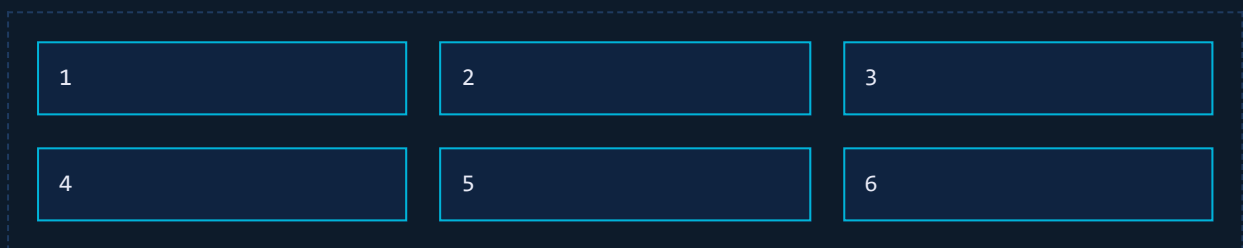
display: grid aur inline-grid

Two-dimensional layout system for rows and columns together

DEFINITION

`display: grid` element ko grid container bana deta hai. Direct children grid items ban jaate hain. Grid rows aur columns dono directions mein layout control karne ke liye best hai.

GRID VISUAL DEMO



● ● ● responsive cards grid

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 1rem;  
}
```

■ grid vs inline-grid

Value	Outer Behavior	Inner Layout	Use
<code>display: grid</code>	Block-level grid container	Grid formatting context	Page layout, cards, dashboards, galleries
<code>display: inline-grid</code>	Inline-level grid container	Grid formatting context	Small inline layouts, compact controls

■ Flexbox vs Grid Decision

Need	Choose	Reason
Only row or only column control	Flexbox	One-dimensional layout simple hota hai
Rows and columns together	Grid	Two-dimensional layout ka direct control milta hai
Unknown number of cards	Grid	auto-fit + minmax responsive card layout bana deta hai
Icon and text align karna	inline-flex	Small component alignment fast and readable

■ **Advanced Point:** Flex aur Grid dono direct children par apply hote hain. Grandchildren ko affect karna ho to nested flex/grid lagana padega.

display: flow-root

Modern clearfix aur block formatting context create karne ka clean method

DEFINITION

`display: flow-root` element ko block-level box banata hai aur uske andar new block formatting context create karta hai. Simple words mein, floats contain ho jaate hain aur margin/floating problems clean ho sakti hain.

■ Float Contain Karna

Purane layouts mein float use hota tha. Float child normal flow se partially bahar behave karta hai, jisse parent height collapse ho sakti hai. `flow-root` parent ko apne floated children contain karne mein help karta hai.

FLOW-ROOT DEMO

float

Yeh parent `display: flow-root` use kar raha hai, isliye floated box parent ke andar properly contained hai. Parent ka border content ke around stay karta hai.

modern clearfix

```
.article-card {  
  display: flow-root;  
}  
  
.article-card img {  
  float: left;  
  margin-right: 1rem;  
}
```

■ Block Formatting Context Kab Useful Hai?

Problem

flow-root Help

Parent floated children ko contain nahi kar raha

Parent height correctly wrap kar sakta hai

Old clearfix hack avoid karna hai

Single property se clean solution

Outer margin collapsing control chahiye

New formatting context margin behavior isolate karta hai

■ **Practical Use:** Aaj ke projects mein floats kam use hote hain, but `flow-root` CSS interviews aur legacy layouts debug karne ke liye important hai.

display: contents

Wrapper ka box remove karna, children ko parent layout mein promote karna

DEFINITION

`display: contents` element ka apna box generate nahi karta. Uske children layout mein aise behave karte hain jaise wrapper exist hi nahi karta, but HTML structure mein wrapper still present hota hai.

contents example

```
.card-list {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}  
  
.wrapper {  
  display: contents;  
}
```

■ Kab Use Kar Sakte Hain?

Kabhi-kabhi HTML mein wrapper semantic ya framework reason se zaroori hota hai, but layout mein us wrapper ka box unwanted hota hai. Aise case mein `display: contents` children ko parent grid/flex mein directly participate karwa sakta hai.

■ Important Limitations

Limitation	Reason
Background/border/padding wrapper par visible nahi hote	Wrapper ka own box generate nahi hota
Accessibility issues historically possible rahe hain	Some browser/screen reader combinations mein semantics affected ho sakte the
Pseudo-elements behavior tricky ho sakta hai	Element box absent hone se layout expectation change hoti hai

■ **Careful:** `display: contents` powerful hai, but normal daily layout ke liye first choice nahi. Use tab karo jab wrapper box remove karna genuinely needed ho.

Table, List and Special Display Values

HTML-like layout behavior CSS display se manually create karna

■ 1. display: list-item

`display: list-item` element ko list item ki tarah render karta hai, including marker/bullet. Default `li` ka display list-item hota hai.

● ● ● custom list item

```
.custom-point {  
  display: list-item;  
  list-style-position: inside;  
  list-style-type: square;  
}
```

■ 2. Table Display Values

CSS mein table-like layout values available hain, jaise `table`, `table-row`, `table-cell`. Aaj ke modern layout mein Flexbox/Grid usually better hote hain, but old email templates, equal height tricks aur legacy code mein yeh values mil sakti hain.

Display Value	Similar HTML Element	Use
<code>table</code>	<code>table</code>	Table formatting context create karta hai
<code>table-row</code>	<code>tr</code>	Table row jaisa behavior
<code>table-cell</code>	<code>td</code>	Cell jaisa behavior, vertical-align useful
<code>table-caption</code>	<code>caption</code>	Caption jaisa behavior
<code>table-header-group</code>	<code>thead</code>	Header group behavior
<code>table-footer-group</code>	<code>tfoot</code>	Footer group behavior

```
.legacy-row {  
  display: table;  
  width: 100%;  
}  
  
.legacy-cell {  
  display: table-cell;  
  vertical-align: middle;  
}
```

■ **Modern Rule:** New projects mein normal layouts ke liye Grid/Flex use karo. Table display values tab useful hain jab actual table-like behavior ya legacy compatibility needed ho.

Advanced Display: Outer and Inner Values

Modern CSS display ko outer box aur inner layout ke combination ki tarah samajhna

ADVANCED CONCEPT

Modern CSS specification ke according display value do parts mein socha ja sakta hai: outer display type aur inner display type. Outer decide karta hai element outside world ke saath kaise behave karega. Inner decide karta hai children ka layout system kya hoga.

■ Outer vs Inner Display

Part	Meaning	Examples
Outer display	Element khud surrounding layout mein block hai ya inline	block, inline
Inner display	Element ke children ka layout mode kya hai	flow, flow-root, flex, grid, table

■ Old Single Values Ka Meaning

Common Value	Modern Meaning	Explanation
block	block flow	Outside block, inside normal flow
inline	inline flow	Outside inline, inside normal flow
inline-block	inline flow-root	Outside inline, inside independent block container
flex	block flex	Outside block, children flex items
inline-flex	inline flex	Outside inline, children flex items
grid	block grid	Outside block, children grid items
inline-grid	inline grid	Outside inline, children grid items

```
.box {  
  display: block flow; /* same idea as block */  
}  
  
.chip {  
  display: inline flex; /* similar to inline-flex */  
}  
  
.layout {  
  display: block grid; /* similar to grid */  
}
```

■ **Compatibility Rule:** Daily coding mein old values like `flex`, `grid`, `inline-flex` still safest and most readable hain. Two-value syntax concept samajhne ke liye important hai.

Display Property aur Box Model

Width, height, margin, padding display ke saath kaise behave karte hain

■ Block Formatting Behavior

Block elements parent ki available width fill karte hain. Agar `width` set karo, to remaining horizontal space margins mein distribute ho sakta hai. Isi wajah se `margin: auto` block element ko center kar sakta hai.

● ● ● block center

```
.container {  
  display: block;  
  width: min(90%, 1100px);  
  margin-inline: auto;  
}
```

■ Inline Box Behavior

Inline elements text line boxes ke andar layout hote hain. Inka width/height directly set nahi hota, kyunki size content aur line-height se aata hai. Agar button-like clickable area chahiye, inline-block ya inline-flex better hai.

Property	Inline	Inline-Block	Block
width	Ignored	Works	Works
height	Ignored	Works	Works
padding-left/right	Works	Works	Works
padding-top/bottom	Visual effect, line layout tricky	Works cleanly	Works cleanly
margin-left/right	Works	Works	Works
margin-top/bottom	No normal layout push	Works	Works

■ **Common Bug:** `a` tag par padding diya but height/width expected nahi aa rahi? Usko `display:`
`inline-block` ya `inline-flex` karo.

Real Projects Mein Display Ka Use

Navbar, cards, mobile menu, center layout aur utility classes

■ 1. Navbar Layout

● ● ● navbar pattern

```
.nav {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
}  
  
.nav-links {  
  display: flex;  
  gap: 1rem;  
}
```

■ 2. Responsive Cards

● ● ● card grid pattern

```
.card-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 1.25rem;  
}
```

■ 3. Mobile Menu Toggle

● ● ● mobile display change

```
.menu-button {  
  display: none;  
}  
  
@media (max-width: 768px) {  
  .menu-button {  
    display: inline-flex;  
  }  
  
  .nav-links {  
    display: none;  
  }  
  
  .nav-links.is-open {  
    display: flex;  
    flex-direction: column;  
  }  
}
```

■ 4. Center Any Box

● ● ● center patterns

```
.center-with-flex {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}  
  
.center-with-grid {  
  display: grid;  
  place-items: center;  
}
```


■ **Real Project Rule:** Layout ke parent par display set hota hai. Agar children align nahi ho rahe, check karo property parent par lagni chahiye ya child par.

CSS Display Property - Complete Summary

Revision ke liye all important display values ek jagah

Display Value	Meaning	Best Use
<code>block</code>	New line, full available width	Containers, sections, cards
<code>inline</code>	Text flow ke andar, content width	Span, links, text formatting
<code>inline-block</code>	Inline outside, block-like sizing	Buttons, badges, chips
<code>none</code>	Element layout se remove	Hide/show UI, mobile menus
<code>flex</code>	Block-level flex container	Navbar, row/column alignment
<code>inline-flex</code>	Inline-level flex container	Icon buttons, pills, compact controls
<code>grid</code>	Block-level grid container	Cards, page layouts, galleries
<code>inline-grid</code>	Inline-level grid container	Small inline two-dimensional layouts
<code>flow-root</code>	New block formatting context	Float clearing, formatting isolation
<code>contents</code>	Wrapper box remove, children promoted	Advanced wrapper/layout fixes
<code>list-item</code>	List marker ke saath item box	Custom list-like elements
<code>table</code> / <code>table-row</code> / <code>table-cell</code>	Table-like formatting	Legacy layouts, table behavior

■ Fast Decision Guide

Question	Choose
Element next line par full width lena chahiye?	block
Element text ke beech same line mein rehna chahiye?	inline
Same line but width/height/padding control chahiye?	inline-block
Direct children row/column mein align karne hain?	flex
Rows and columns dono control karne hain?	grid
Element completely hide karna hai, space bhi nahi chahiye?	display: none
Element hide karna hai but space preserve chahiye?	visibility: hidden
Fade animation chahiye?	opacity + visibility + pointer-events
Float parent collapse fix karna hai?	flow-root
Wrapper ka box remove karna hai?	contents

■ Best Practices - Yaad Rakhna

DO

Layout parent par display set karo.
 Buttons ke liye inline-flex/inline-block use karo.
 Cards ke liye grid useful hota hai.
 Navbar ke liye flex clean hota hai.
 Hidden UI mein accessibility test karo.
 display value ka computed style inspect karo.

AVOID

Inline elements par width/height expect karna.
 display none ko fade animation ke liye use karna.
 Har layout ko inline-block se banana.
 contents blindly use karna.
 Flex/Grid properties child par lagakar result expect karna.
 Visibility aur opacity ko display none samajhna.

> NEXT TOPICS TO COVER

Flexbox Deep Dive

CSS Grid

Positioning

Overflow

Z-index

Responsive Navbar

Media Queries

Layout Debugging

Made with ❤️ for Web Development Course learners

Shauray ke saath CSS seekhte raho 🚀 | Topic 07 - CSS Display Property